Micro Processor Fundamentals

String Instructions in 80x86 Microprocessors

Dr. Hesham El Zouka

String Instructions

The 80x86 microprocessor family is equipped with special instructions for handling string operations. A string is a series of data words or bytes stored in consecutive memory locations. String operations include moving, scanning, and comparing data.

String Instruction:

Byte Transfer: The SI or DI register is incremented or decremented by 1.
Word Transfer: The SI or DI register is incremented or decremented by 2.
Double-Word Transfer: The SI or DI register is incremented or decremented by 4.

String Instructions – Direction Flag

The Direction Flag (DF):

The Direction Flag determines the operation mode for the SI and DI registers during string operations:

- D = 0: Enables auto-increment of the registers.
- D = 1: Enables auto-decrement of the registers.

This flag is exclusively used with string instructions.

Mnemonic	Meaning	Format	Operation	Flags Affected
CLD	Clear DF	CLD	(DF) 0	DF
STD	Set DF	STD	(DF) 1	DF

Sets and Clears the Direction Flag (DF):

- **STD: Sets the Direction Flag (DF = 1).**
- **CLD: Clears the Direction Flag (DF = 0).**

String Instruction Set

Mnemonic	Meaning	Format	Operation	Flags Affected
MOVS	Move string	MOVSB/ MOVSW	Copies data from **DS: [SI]** to **ES: [DI]**; updates **SI** and **DI** based on direction flag (SI) = (SI)±1 or 2 (DI) = (DI)±1 or 2	None
CMPS	Compare string	CMPSB/ CMPSW	Compares data at **DS: [SI]** to **ES: [DI]** and updates flags accordingly (SI) = (SI)±1 or 2 (DI) = (DI)±1 or 2	CF, PF, AF, ZF, SF, OF
SCAS	Scan string	SCASB/ SCASW	Compares **AL / AX** with **ES: [DI]** and updates flags accordingly (AL or AX) - (ES)0+(DI) (DI) = (DI)±1 or 2	CF, PF, AF, ZF, SF, OF
LODS	load string	LODSB/ LODSW	Loads data from **DS:[SI]** into **AL / AX** (AL or AX) -(DS)0+(SI) (SI) = (SI)±1 or 2	None
STOS	Store string	STOSB/ STOSW	Stores **AL/AX** into **ES: [DI]** (ES)0+(DI)-(AL or AX)±1 or2 (DI) = (DI)±1 or 2	None

Example: MOVSB (Move String Byte)

```
MOV AX, @DATA
MOV DS, AX
MOV ES, AX
MOV SI, offset Array1
MOV DI, offset Array2
CLD
MOV CX, 5
NXTPT: MOVSB
LOOP NXTPT
HLT
```

Repeat String Instructions (REP)

The **REP** prefix is used to repeat string operations for processing arrays or sequences of data. It works by automatically decrementing the **CX** register (loop counter) and executing the associated instruction until **CX = 0**.

Prefix	Used with	Meaning
REP	MOVS STOS	Repeats the operation until CX = 0.
REPE / REPZ	CMPS SCAS	Repeat the operation while CX≠0 and the Zero flag (ZF) is set (strings are equal). i.e. CX≠0 and ZF=1
REPNE / REPNZ	CMPS SCAS	Repeat the operation while CX≠0 and the Zero flag (ZF) is clear (strings are not equal). i.e. CX≠0 and ZF=0

Example: Find and Replace Characters in a String

Write a program that scans the string "Dr. Hisham" and replaces every occurrence of the letter "i" with the letter "e".

```
DATA1  DB  'Dr. Hisham' , '$'     ;Define the input string with a terminator '$'

.CODE
  :::
      MOV ES, DS                  ; Set ES to point to the data segment
      CLD                         ; Clear the Direction Flag (DF=0) for auto-increment
      MOV DI, OFFSET DATA1        ; Load the offset of DATA1 into DI
      MOV CX, 10                  ; Set CX to the number of characters to scan (10)
      MOV AL, ' i '               ; Load the character to search for (' i ') into AL

 REPNE SCASB                      ; Scan the string for the character in AL
      JNE OVER                    ; If not found (ZF = 0), jump to OVER
      DEC DI                      ; Adjust DI to point back to the matched character
      MOV BYTE PTR [DI] , ' e '   ; Replace ' i ' with ' e '

 OVER:
       MOV AH, 09                 ; Prepare to display the string
       MOV DX, OFFSET DATA1       ; Load the address of the string into DX
       INT 21H                    ; DOS interrupt to display the string
  :::
```